

SecretHunter Android : une plateforme web d'analyse statique pour la détection des secrets, endpoints et risques dans les APK Android

Hidaya Benchelha, Asma Bensassi Nour, Ikram Laabouki, Halima Badr

Encadré par : Mr. Mohamed LACHGAR

École Nationale des Sciences Appliquées de Marrakech, Université Cadi Ayyad, Maroc

Abstract

SecretHunter Android est une plateforme académique d'analyse statique des APK Android. Elle permet d'importer un APK autorisé, d'extraire ses fichiers internes, d'identifier les secrets codés en dur, de détecter les endpoints réseau, de calculer un score de risque et de produire des rapports JSON/HTML. L'architecture repose sur une interface React/Vite et un backend Flask qui orchestre JADX, Apktool, les scanners, le moteur de scoring et un moteur de recommandations. Le processus complet est modélisé afin d'en faciliter la compréhension, la reproduction et la démonstration. La version v1.0.0 a été testée sous Windows avec Python, Node.js, Java/JDK, JADX et Apktool. Sur le corpus de validation composé de DIVA, OWASP UnCrackable Level 2 et InsecureBankv2, le prototype a analysé respectivement 3 482, 1 972 et 10 680 fichiers, détecté 6, 0 et 24 secrets, et produit des scores de 68/100, 20/100 et 72/100. Ces résultats montrent que le pipeline est opérationnel, tout en soulignant la nécessité d'améliorer le filtrage des faux positifs, notamment pour les endpoints.

Keywords: Sécurité Android, Analyse APK, Analyse statique, Secrets codés en dur, Endpoints, Flask, React, JADX, Apktool, DevSecOps

Métadonnées

Nr.	Description	Information
C1	Current code version	v1.0.0, version de soumission académique. Tag GitHub stable : v1.0.0.
C2	Permanent link to code/repository	https://github.com/halima200431/secret-hunter-android.git ; tag permanent recommandé : https://github.com/halima200431/secret-hunter-android/releases/tag/v1.0.0 .
C3	Permanent link to reproducible capsule	Reproductibilité locale via <code>requirements.txt</code> , <code>package-lock.json</code> , scripts <code>scripts/</code> et futur <code>docker-compose.yml</code> . Capsule Docker publique : non archivée dans la version actuelle.
C4	Legal code license	MIT License, à confirmer par la présence du fichier <code>LICENSE</code> dans le dépôt.
C5	Code versioning system used	Git/GitHub avec commits, historique de collaboration et tag de version.
C6	Software code languages, tools, and services	Python, Flask, React, Vite, Tailwind CSS, JavaScript, JSON, HTML, JADX, Apktool, PowerShell.
C7	Compilation requirements	Python 3, Node.js/npm, Java/JDK, JADX, Apktool, Windows ou Linux. Dépendances dans <code>requirements.txt</code> et <code>package-lock.json</code> .
C8	Developer documentation	README principal et dossier <code>docs/</code> : installation, API, exemples d'APKs, structure des rapports, reproduction des tests.
C9	Support email	gcdste0@gmail.com

TABLE 1 – Métadonnées complètes du code de SecretHunter Android.

1. Motivation et importance

Les applications Android contiennent fréquemment des informations sensibles dans leur code ou leurs fichiers de configuration : clés API, tokens d'authentification, mots de passe, URLs backend, adresses IP internes, paramètres Firebase, endpoints HTTP et chemins d'API. Après décompilation, ces informations peuvent être lisibles et faciliter la compréhension de l'architecture applicative ou la préparation d'attaques ciblées. Un attaquant peut ainsi identifier un serveur backend, réutiliser un token, tester une clé API ou déduire la logique métier d'une application.

Dans un contexte pédagogique, l'analyse statique des APKs est un moyen efficace pour comprendre comment une application mobile est structurée et comment des erreurs de configuration peuvent exposer des données sensibles. Les standards OWASP MASVS et MASTG recommandent de vérifier la protection des secrets, la sécurité des communications, les configurations réseau et les mécanismes de stockage local [1, 2]. SecretHunter Android s'inscrit dans cette démarche en proposant un outil simple à déployer et à expliquer.

L'objectif n'est pas de remplacer des plateformes avancées comme MobSF, mais de construire une chaîne d'analyse claire et modulaire : upload de l'APK, validation, extraction, décompilation, scan, scoring, rapport et recommandations. Cette approche permet à chaque membre de l'équipe de comprendre une partie précise du pipeline tout en contribuant à une plateforme cohérente.

2. Description globale de SecretHunter Android

SecretHunter Android est une application web full-stack. Le frontend, développé avec React/-Vite, fournit une interface claire pour l'import d'APK et la consultation des résultats. Le backend, développé avec Flask, expose une API REST locale qui orchestre le pipeline d'analyse. Les fichiers sont organisés localement dans des répertoires séparés : `uploads/` pour les APKs originaux, `extracted/` pour les fichiers décompilés, `reports/` pour les rapports et `logs/` pour les traces.

La plateforme se concentre sur trois familles de résultats : les secrets, les endpoints et le score de risque. Les secrets correspondent aux tokens, clés API, mots de passe, secrets clients et clés privées. Les endpoints correspondent aux URLs, domaines, adresses IP et chemins de services backend. Le score de risque fournit une lecture rapide du niveau d'exposition de l'application.

2.1. Objectifs fonctionnels

Les objectifs fonctionnels sont les suivants :

- permettre l'import d'un fichier APK autorisé depuis une interface web ;
- valider le fichier reçu avant analyse ;
- extraire les ressources et décompiler le code avec des outils standards ;
- détecter automatiquement les secrets codés en dur ;
- identifier les endpoints réseau potentiellement exposés ;
- calculer un score global de risque ;
- produire des recommandations compréhensibles ;
- générer des rapports JSON et HTML exploitables.

2.2. Objectifs pédagogiques

Le projet vise également des objectifs pédagogiques :

- comprendre la structure interne d'un APK ;
- apprendre le rôle de JADX et Apktool dans l'analyse statique ;
- manipuler des expressions régulières pour la détection de secrets ;
- interpréter des faux positifs et des résultats critiques ;
- relier les résultats techniques à des recommandations de sécurité ;
- apprendre à organiser un projet full-stack collaboratif avec GitHub.

3. Architecture logicielle

3.1. Vue d'ensemble

La Figure 2 représente l'architecture globale. Le frontend envoie une requête POST `/api/analyze` au backend. Le backend crée un identifiant d'analyse, enregistre l'APK, exécute l'extraction, lance les scanners, calcule le score et renvoie les résultats au frontend. Le frontend affiche ensuite les résultats dans le dashboard, les pages secrets, endpoints, analyse IA et rapport.

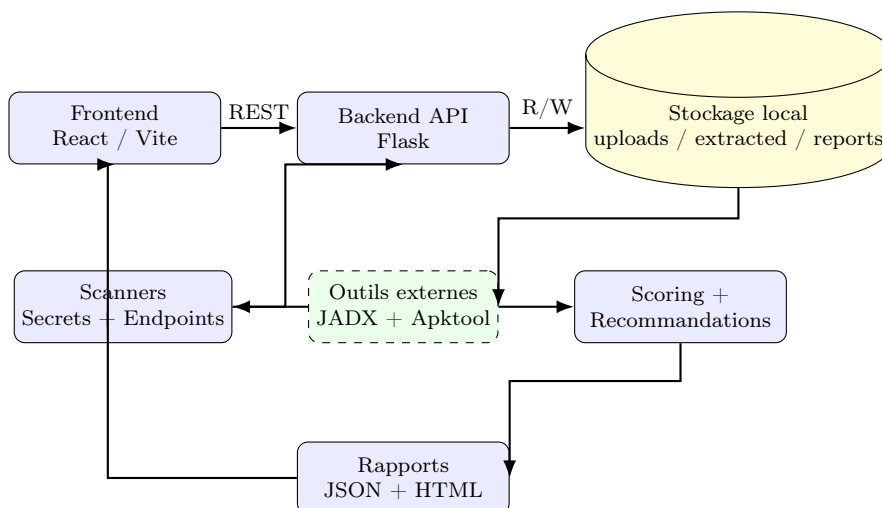


FIGURE 2 – Schéma global de l'architecture de SecretHunter Android.

3.2. Découpage frontend

Le frontend est organisé en pages et composants. Les pages principales sont : `Upload.jsx`, `Dashboard.jsx`, `Secrets.jsx`, `Endpoints.jsx`, `AIAnalysis.jsx`, `Report.jsx` et `About.jsx`. Les composants réutilisables incluent `Sidebar.jsx`, `Navbar.jsx`, `StatCard.jsx`, `RiskBadge.jsx`, `FindingsTable.jsx`, `RecommendationCard.jsx` et `ProgressSteps.jsx`.

3.3. Découpage backend

Le backend regroupe les services, scanners, règles, schémas et générateurs de rapports. Ce découpage facilite la maintenance : le scanner de secrets peut être modifié sans impacter la génération du rapport, et le scoring peut évoluer sans modifier l'upload.

TABLE 2 – Rôle des principaux modules backend.

Module	Responsabilité
app.py	Crée l'application Flask, configure CORS, dossiers et routes principales.
routes_analysis.py	Reçoit l'APK, crée le job, lance le pipeline, expose status/results.
apk_service.py	Orchestration de l'extraction et normalisation du résultat.
apk_extractor.py	Appelle JADX/Apktool et recense les fichiers extraits.
secret_scanner.py	Applique les règles de <code>secret_patterns.json</code> et masque les valeurs sensibles.
endpoint_scanner.py	Détecte URLs, domaines et IP, puis retourne fichier, ligne, type et risque.
risk_scoring.py	Calcule scores unitaires, score global et distribution des risques.
ai_service.py	Génère un résumé et des recommandations à partir des résultats.
report_generator.py	Produit les rapports JSON/HTML associés à l'identifiant d'analyse.

4. Processus d'analyse et contrats API

4.1. Contrats API

Les échanges entre le frontend et le backend sont basés sur des objets JSON. Le champ `analysisId` relie l'upload, le statut et les résultats. Le tableau suivant résume les routes principales.

TABLE 3 – Contrats API principaux utilisés par le prototype.

Route	Méthode	Réponse principale
/api/analyze	POST	Reçoit un <code>.apk</code> . Retourne <code>status</code> , <code>message</code> , <code>analysisId</code> , <code>apkName</code> .
/api/analysis/<id>/status	GET	Retourne l'état du job : <code>pending</code> , <code>running</code> , <code>completed</code> , <code>failed</code> .
/api/analysis/<id>/results	GET	Retourne le résultat final : compteurs, secrets, endpoints, score, risque, recommandations, rapports.
/	GET	Vérifie que le backend est démarré et retourne la version.

4.2. Structure JSON du résultat final

Le résultat final suit une structure stable :

```
{
  "apkName": "app-debug.apk",
  "filesAnalyzed": 10680,
  "secretsCount": 24,
  "endpointsCount": 2352,
  "globalScore": 72,
  "globalRisk": "High",
  "riskDistribution": [...],
  "secrets": [...],
  "endpoints": [...],
  "aiSummary": "...",
  "recommendations": [...],
  "errors": [],
  "reports": {...}
}
```

Cette structure est importante car elle permet d’afficher les mêmes résultats dans plusieurs pages du frontend et de générer des rapports reproductibles. Elle prépare également l’intégration future avec CI/CD ou SARIF.

4.3. Diagramme BPMN du pipeline

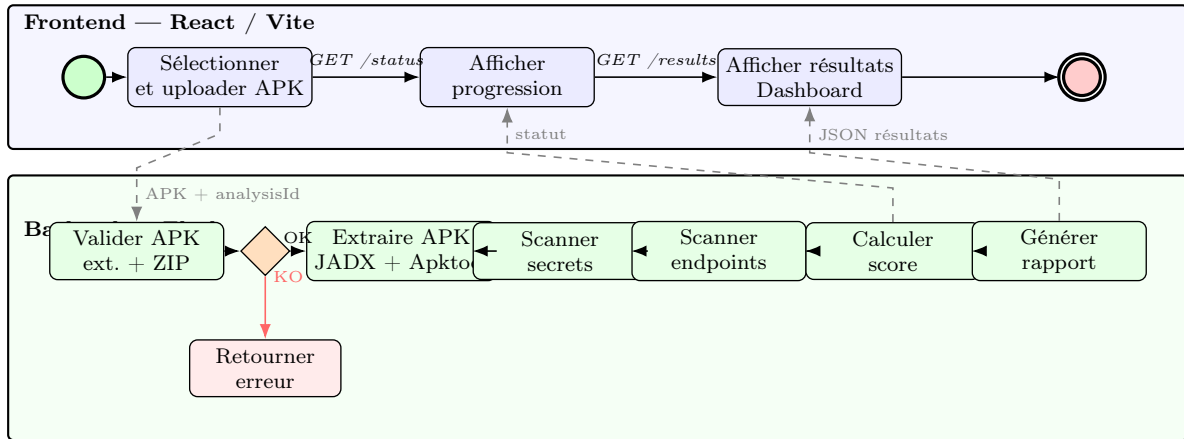


FIGURE 4 – Diagramme de processus du pipeline SecretHunter Android.

5. Sécurité de l’upload et gestion des fichiers

La sécurité de l’upload est essentielle, car l’application manipule des fichiers APK potentiellement volumineux et des résultats d’analyse sensibles. Le backend applique plusieurs contrôles : vérification de l’extension, validation ZIP, limite de taille, création d’un `analysisId` unique et séparation des artefacts par dossier.

Les répertoires sensibles ne doivent pas être poussés dans GitHub. Le fichier `.gitignore` doit exclure `uploads/`, `extracted/`, `reports/`, `logs/`, `node_modules/`, `.env/` et `backend/models/` si un modèle IA local est ajouté.

TABLE 4 – Mesures de sécurité appliquées à l’upload et au stockage local.

Mesure	Objectif
Extension <code>.apk</code>	Rejeter les fichiers qui ne correspondent pas au format attendu.
Validation ZIP	Vérifier que l’APK possède une structure exploitable avant extraction.
Limite de taille	Éviter les fichiers trop volumineux et les risques de saturation disque.
<code>analysisId</code>	Isoler chaque analyse dans un dossier unique.
Masquage des secrets	Éviter de réexposer les valeurs sensibles dans les rapports.
<code>.gitignore</code>	Empêcher la publication accidentelle d’artefacts sensibles.

6. Extraction et décompilation APK

L’extraction est la première étape technique du pipeline. Elle transforme un APK opaque en un ensemble de fichiers analysables. Apktool reconstruit les ressources Android, les fichiers XML et le manifeste. JADX permet d’obtenir des sources Java lisibles à partir des fichiers

DEX. Les deux outils sont complémentaires : Apktool est utile pour les ressources et la structure Android, tandis que JADX est utile pour comprendre le code applicatif.

Le backend conserve également l'APK original afin de faciliter la traçabilité. Le nombre de fichiers analysés est calculé après extraction afin de fournir une métrique de couverture au dashboard. Cette métrique n'est pas une mesure de sécurité, mais elle permet de vérifier que l'extraction a réellement produit des artefacts.

TABLE 5 – Rôle des outils d'extraction utilisés.

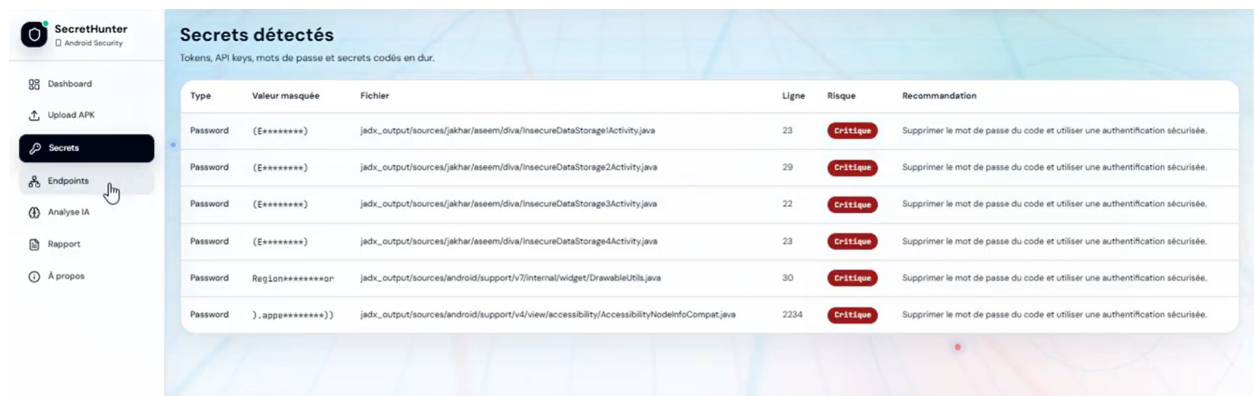
Outil	Rôle dans SecretHunter Android
JADX	Décompiler les fichiers DEX en sources Java lisibles et faciliter l'analyse du code.
Apktool	Extraire les ressources, XML, manifestes et structure interne de l'APK.
Python	Orchestrer les commandes, capturer les erreurs et normaliser les chemins.
Flask	Exposer le résultat d'extraction au pipeline d'analyse via API.

7. Détection des secrets

7.1. Principe

Le scanner de secrets parcourt les fichiers texte extraits et applique des règles d'expressions régulières définies dans `backend/rules/secret_patterns.json`. Les règles ciblent les clés API, tokens Bearer, JWT, mots de passe, secrets clients, clés privées et identifiants de base de données. Pour chaque occurrence, le scanner enregistre le type, la valeur masquée, le fichier, la ligne, le risque et la recommandation associée.

Le masquage est essentiel. Un rapport de sécurité ne doit pas réexposer la valeur complète d'un secret détecté. Le scanner conserve donc une version masquée comme `AIzaSyFa*****` ou `Bearer eyJ*****`. Cette pratique protège le rapport et rend sa diffusion moins risquée.



The screenshot shows the 'Secrets détectés' section of the SecretHunter Android Security dashboard. It displays a table of detected secrets with columns for Type, Valeur masquée, Fichier, Ligne, Risque, and Recommandation. The table lists six entries, all categorized as 'Password' with a 'Critique' risk level. The recommendations for all entries are to 'Supprimer le mot de passe du code et utiliser une authentification sécurisée.' The dashboard also includes a sidebar with navigation options: Dashboard, Upload APK, Secrets (selected), Endpoints, Analyse IA, Rapport, and À propos.

Type	Valeur masquée	Fichier	Ligne	Risque	Recommandation
Password	(E*****)	jadx_output/sources/jakhar/aseem/diva/insecureDataStorage1Activity.java	23	Critique	Supprimer le mot de passe du code et utiliser une authentification sécurisée.
Password	(E*****)	jadx_output/sources/jakhar/aseem/diva/insecureDataStorage2Activity.java	29	Critique	Supprimer le mot de passe du code et utiliser une authentification sécurisée.
Password	(E*****)	jadx_output/sources/jakhar/aseem/diva/insecureDataStorage3Activity.java	22	Critique	Supprimer le mot de passe du code et utiliser une authentification sécurisée.
Password	(E*****)	jadx_output/sources/jakhar/aseem/diva/insecureDataStorage4Activity.java	23	Critique	Supprimer le mot de passe du code et utiliser une authentification sécurisée.
Password	Region*****gr	jadx_output/sources/android/support/v7/internal/widget/DrawableUtils.java	30	Critique	Supprimer le mot de passe du code et utiliser une authentification sécurisée.
Password	).appe*****)	jadx_output/sources/android/support/v4/view/accessibility/AccessibilityNodeInfoCompat.java	2234	Critique	Supprimer le mot de passe du code et utiliser une authentification sécurisée.

7.2. Gestion des faux positifs

Certains mots ressemblent à des secrets sans être réellement sensibles. Les contextes `test`, `example`, `dummy`, `placeholder` ou `sample` peuvent indiquer des données de démonstration.

Le scanner peut réduire le score ou ignorer certaines valeurs selon le contexte. Cette logique n'est pas parfaite mais elle limite les résultats inutiles.

TABLE 6 – Exemples de catégories de secrets recherchées.

Catégorie	Exemples	Risque principal
Clé API	Google API Key, Firebase Key	Utilisation abusive de services externes.
Token	Bearer Token, JWT	Réutilisation pour accéder à une ressource protégée.
Mot de passe	<code>password=...</code>	Compromission directe d'un compte ou service.
Secret client	<code>client_secret</code>	Abus d'un flux OAuth ou d'une intégration backend.
Clé privée	Bloc PEM	Compromission cryptographique forte.
Identifiant DB	<code>db_password</code>	Accès non autorisé à une base de données.

8. Détection des endpoints

Le scanner d'endpoints recherche les URLs, domaines et adresses IP dans les fichiers extraits. Cette information est utile pour comprendre les communications potentielles de l'application. Les endpoints peuvent révéler des environnements de test, des URLs HTTP non chiffrées, des domaines backend, des chemins API ou des IP internes.

Cependant, cette détection produit un grand nombre de faux positifs. Des chaînes comme `http://schemas.android.com/apk/res/android`, `java.io` ou `android.app` peuvent être détectées alors qu'elles ne représentent pas un endpoint backend exploitable. Dans la version actuelle, ces éléments sont classés en risque faible. Une version améliorée devra les catégoriser explicitement comme *faux positifs techniques*.

TABLE 7 – Catégories d'endpoints et interprétation.

Catégorie	Interprétation
URL HTTPS	Endpoint potentiellement sécurisé, à vérifier selon l'usage.
URL HTTP	Communication non chiffrée ; risque plus élevé si données sensibles.
Domaine	Surface backend potentielle ou service externe.
Adresse IP	Peut révéler un serveur interne, de test ou de production.
Namespace Android	Faux positif technique, à filtrer.
Import Java/Android	Faux positif technique, à filtrer.

The screenshot shows the 'Endpoints détectés' section of the Secrethunter tool. The interface includes a sidebar with navigation options: Dashboard, Upload APK, Secrets, Endpoints (selected), Analyse IA, Rapport, and À propos. The main area displays a table of detected endpoints with columns for URL / Domaine / IP, Type, Protocole, Fichier, Ligne, and Risque. All listed endpoints are marked as 'Faible' (Low) risk.

URL / Domaine / IP	Type	Protocole	Fichier	Ligne	Risque
http://payatu.com	URL	HTTP	apktool_output\smali\jakhar\aseem\diva\APICreds2Activity.smali	101	Faible
http://payatu.com	URL	HTTP	jdkx_output\sources\jakhar\aseem\diva\APICreds2Activity.java	27	Faible
http://schenas.android.com/apk/res/android	URL	HTTP	apktool_output\AndroidManifest.xml	2	Faible
http://schenas.android.com/apk/res/android	URL	HTTP	apktool_output\res\anim\abc_fade_in.xml	3	Faible
http://schenas.android.com/apk/res/android	URL	HTTP	apktool_output\res\anim\abc_fade_out.xml	3	Faible
http://schenas.android.com/apk/res/android	URL	HTTP	apktool_output\res\anim\abc_grow_fade_in_from_bottom.xml	3	Faible
http://schenas.android.com/apk/res/android	URL	HTTP	apktool_output\res\anim\abc_popup_enter.xml	3	Faible
http://schenas.android.com/apk/res/android	URL	HTTP	apktool_output\res\anim\abc_popup_exit.xml	3	Faible
http://schenas.android.com/apk/res/android	URL	HTTP	apktool_output\res\anim\abc_shrink_fade_out_from_bottom.xml	3	Faible

9. Scoring du risque

Le score global n'est pas calculé uniquement à partir du nombre total de résultats. Il combine deux informations complémentaires : le risque le plus grave observé et le niveau de risque moyen de l'ensemble des résultats. Cette logique reflète une règle importante en sécurité : une seule faille critique peut suffire à rendre l'application dangereuse, mais un grand nombre de problèmes moyens peut aussi représenter un risque global.

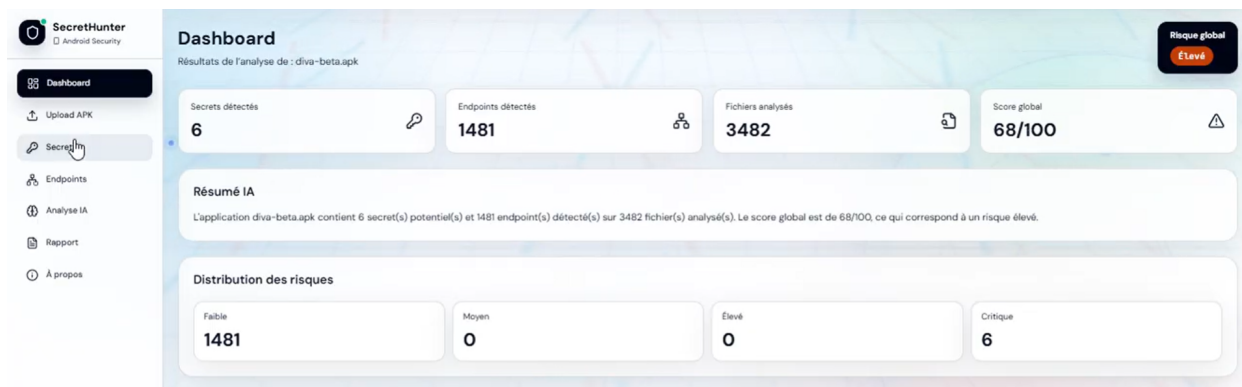
$$\text{global_score} = 0,65 \times \text{score_max} + 0,35 \times \text{score_moyen}$$

Le poids de 65% donné à **score_max** met la priorité sur le problème le plus dangereux. Le poids de 35% donné à **score_moyen** permet de prendre en compte la densité globale des failles. Cette combinaison évite deux extrêmes : une moyenne seule pourrait minimiser une faille critique isolée, tandis qu'un maximum seul ignorerait la quantité totale de problèmes.

TABLE 8 – Seuils de classification du score global.

Intervalle du score	Niveau de risque
0–30	Low
31–60	Medium
61–85	High
86–100	Critical

Les scores unitaires des secrets et endpoints sont calculés par règles. Le score augmente fortement pour les types sensibles comme private key, password, token ou API key. Il peut être réduit si le contexte ressemble à un faux positif ou à une valeur de démonstration.



10. Analyse IA et recommandations

La page *Analyse IA* transforme les résultats techniques en résumé compréhensible et en recommandations de correction. Dans la version v1.0.0, ce module est principalement basé sur des règles : il analyse le nombre de secrets, les types détectés, le nombre d'endpoints, le score global et le niveau de risque. Il génère ensuite un résumé et des recommandations prioritaires.

Une extension future peut intégrer un modèle local Hugging Face tel que FLAN-T5 pour générer des recommandations plus dynamiques. Dans ce cas, les résultats d'analyse sont transformés en prompt structuré, puis le modèle produit un texte de synthèse. Un fallback par règles doit rester disponible afin de garantir la stabilité du prototype si le modèle n'est pas installé, trop lent ou produit une sortie non structurée.

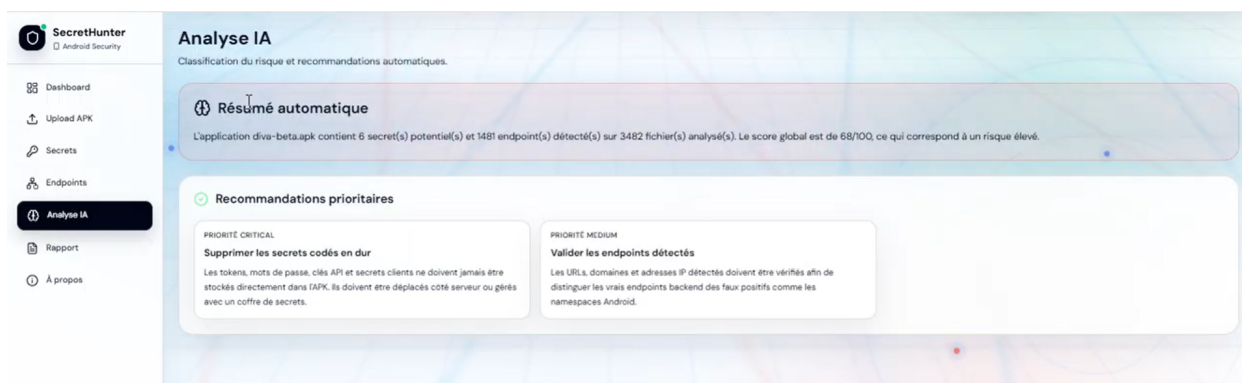


TABLE 9 – Différence entre moteur de règles et modèle IA local.

Critère	Moteur de règles	Modèle IA local
Stabilité	Très stable	Dépend du modèle et du PC
Reproductibilité	Élevée	Variable selon versions et cache
Qualité rédactionnelle	Correcte mais répétitive	Plus flexible et naturelle
Dépendances	Légères	<code>torch</code> , <code>transformers</code> , modèle local
Sécurité données	Locale	Locale si modèle téléchargé
Usage conseillé	Prototype stable	Extension future contrôlée

11. Rapports JSON et HTML

Le générateur de rapports produit un fichier JSON et un fichier HTML pour chaque analyse. Le JSON est utile pour l'automatisation, l'intégration future CI/CD et la réutilisation des résultats. Le HTML est utile pour la lecture humaine et la démonstration. Les exports PDF et SARIF sont prévus comme extensions.

Le rapport doit éviter d'afficher les secrets en clair. Les valeurs sensibles sont masquées. Les rapports sont stockés dans `reports/<analysisId>/`. Cette organisation facilite la traçabilité des analyses et permet de récupérer l'historique localement.

12. Scénario reproductible

Le scénario de base consiste à lancer le backend, lancer le frontend, puis analyser un APK de test. Exemple sous Windows PowerShell :

```
cd secret-hunter-android
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
.\.venv\Scripts\python.exe -m backend.app

cd frontend
npm.cmd install
npm.cmd run dev
```

L'utilisateur ouvre ensuite <http://localhost:5173>, importe par exemple `diva-beta.apk`, puis récupère le rapport JSON dans `reports/<analysisId>/report.json`. Une réponse minimale attendue contient :

```
{"apkName":"diva-beta.apk","filesAnalyzed":3482,
 "secretsCount":6,"endpointsCount":1481,
 "globalScore":68,"globalRisk":"High"}
```

13. Comparaison avec les outils existants

13.1. Méthodologie de comparaison

SecretHunter Android est comparé à MobSF et Yaazhini/Vooki sur des critères fonctionnels. MobSF est un framework complet d'analyse mobile, tandis que Yaazhini/Vooki est orienté vers le scan APK/API. Les scores numériques ne sont pas directement comparables, car chaque outil utilise sa propre logique de scoring. La comparaison doit donc être lue comme une comparaison de couverture et de fonctionnalités.

TABLE 10 – Comparaison fonctionnelle avec des outils représentatifs d’analyse Android.

Critère	SecretHunter v1.0.0	Android	MobSF	Yaazhini / Vooki
Type d’outil	Prototype web académique		Framework mobile complet	Scanner APK/API
Plateformes	Android		Android, iOS, Windows Mo- bile	Android
Entrées	APK		APK, IPA, APPX, code source	APK, API REST
Analyse statique	Oui		Oui	Oui
Analyse dynamique	Non		Oui	Partielle selon version
Extraction APK	JADX, Apktool		Intégrée	Intégrée
Secrets codés en dur	Oui, règles regex		Oui	Oui
Endpoints réseau	Oui, faux positifs possibles		Oui	Oui
Scoring	Score /100 local		Score de sécurité	Classification de risques
API d’automatisation	API Flask locale		API REST complète	Non clairement documentée
Exports	JSON/HTML, PDF/SARIF prévus		Rapports + API	Rapports selon version
CI/CD et Docker	Prévu		Possible, Docker disponible	Dépend de la version
Maturité	Prototype pédagogique		Outil mature	Outil spécialisé

13.2. Analyse comparative avec MobSF

La comparaison avec MobSF permet de mieux positionner SecretHunter Android. MobSF est un framework mature d’analyse mobile couvrant un large périmètre : permissions, certificats, manifeste, trackers, composants Android, bibliothèques tierces, configuration réseau et analyse dynamique. SecretHunter Android adopte une approche plus ciblée : il se concentre principalement sur la détection des secrets codés en dur, des endpoints réseau, la localisation des résultats et la compréhension pédagogique du risque.

Ainsi, les deux outils ne doivent pas être vus comme concurrents directs, mais comme complémentaires. MobSF offre une vision globale de la sécurité mobile, tandis que SecretHunter Android met l’accent sur une lecture simple, rapide et orientée secrets/endpoints. Cette spécialisation est intéressante dans un contexte académique, car elle permet de comprendre clairement le lien entre un fichier décompilé, un secret détecté, un score de risque et une recommandation.

TABLE 11 – Positionnement général de SecretHunter Android par rapport à MobSF.

Critère	SecretHunter Android	MobSF
Nature de l’outil	Prototype académique spécialisé dans les secrets, endpoints et scoring pédagogique.	Framework mature d’audit mobile avec couverture large.
Objectif principal	Comprendre rapidement les expositions sensibles présentes dans un APK.	Réaliser une évaluation complète de la sécurité mobile.
Type d’analyse	Analyse statique ciblée.	Analyse statique et dynamique.
Public visé	Étudiants, développeurs débutants, démonstrations pédagogiques.	Auditeurs sécurité, pentesters, équipes Dev-SecOps.
Lecture des résultats	Dashboard simple : secrets, endpoints, fichiers analysés, score et recommandations.	Rapport très détaillé, plus riche mais parfois plus complexe à interpréter.
Valeur ajoutée	Simplicité, localisation des secrets, score clair, interface adaptée à l’apprentissage.	Couverture complète, maturité, richesse des catégories d’analyse.
Limite principale	Faux positifs possibles sur les endpoints et absence d’analyse dynamique.	Outil plus lourd et moins centré sur l’explication progressive des secrets.

Les tendances globales observées convergent avec MobSF sur les trois APKs. DIVA et InsecureBankv2 sont classés comme risqués par les deux outils, tandis qu’UnCrackable Level 2 reste moins critique. Cette convergence montre que SecretHunter Android fournit une évaluation cohérente malgré un périmètre plus réduit.

TABLE 12 – Comparaison des tendances de risque observées.

APK	SecretHunter Android	MobSF	Convergence	Interprétation
DIVA	High, score 68/100	High, score 36/100, grade C	Oui	Les deux outils confirment que l'application présente un risque élevé. SecretHunter met surtout en avant les secrets et endpoints détectés.
UnCrackable Level 2	Low, score 20/100	Low, score 65/100, grade A	Oui	Les deux outils indiquent un risque global faible. L'APK sert surtout de challenge de reverse engineering et ne contient pas de secret détecté.
InsecureBankv2	High, score 72/100	High, score 31/100, grade C	Oui	Les deux outils considèrent l'application comme risquée. SecretHunter valorise fortement les 24 secrets détectés.

Les scores numériques ne doivent pas être comparés directement, car les deux outils ne calculent pas le risque de la même manière. MobSF prend en compte davantage de catégories comme les permissions, le certificat, le manifeste, les trackers et la configuration générale. SecretHunter Android, lui, donne plus d'importance aux secrets et aux endpoints, car son objectif est de détecter les informations sensibles directement visibles après décompilation.

TABLE 13 – Lecture des différences de score entre SecretHunter Android et MobSF.

APK	Score SecretHunter	Score MobSF	Explication de l'écart
DIVA	68/100	36/100	SecretHunter donne un poids important aux secrets et endpoints détectés. MobSF calcule un score plus global incluant d'autres catégories de sécurité.
UnCrackable Level 2	20/100	65/100	SecretHunter classe l'APK comme faible car aucun secret n'a été détecté. MobSF évalue plus largement la structure de l'application, le manifeste et les configurations.
InsecureBankv2	72/100	31/100	SecretHunter augmente fortement le score à cause des 24 secrets détectés. MobSF confirme également un risque élevé, mais avec sa propre formule de notation.

La force de SecretHunter Android apparaît surtout dans la lisibilité des résultats. L'utilisateur voit immédiatement le nombre de secrets, le nombre d'endpoints, le score global, le niveau de risque et les recommandations. Cette présentation rend l'outil très adapté à une démonstration académique ou à une première sensibilisation à la sécurité Android.

TABLE 14 – Valeur ajoutée de SecretHunter Android dans le cadre du projet.

Élément valorisé	Apport de SecretHunter Android
Spécialisation secrets/endpoints	L'outil se concentre sur les informations sensibles directement exploitables après décompilation : clés API, tokens, mots de passe, domaines, URLs et IP.
Lisibilité pédagogique	Les résultats sont présentés sous forme de dashboard simple avec des indicateurs compréhensibles : secrets détectés, endpoints, fichiers analysés, score global et risque.
Localisation des résultats	Les findings sont associés à des fichiers et à des lignes, ce qui facilite la compréhension et la correction.
Scoring adapté au projet	Le score donne un poids fort aux problèmes critiques, notamment les secrets, tout en prenant en compte l'état général de l'application.
Interface web simple	Contrairement à un outil plus lourd, SecretHunter Android peut être présenté facilement devant un jury et utilisé comme démonstrateur pédagogique.
Architecture modulaire	Chaque module peut être amélioré séparément : extraction, scanner de secrets, scanner d'endpoints, scoring, IA, rapports.
Réutilisation académique	Le projet peut servir de base à d'autres travaux : analyse dynamique, export SARIF, CI/CD, filtrage des faux positifs, modèle IA local.

TABLE 15 – Complémentarité entre SecretHunter Android et MobSF.

Besoin	Outil le plus adapté	Justification
Comprendre les secrets exposés	SecretHunter Android	L'outil est centré sur les secrets et donne une lecture rapide des informations sensibles détectées.
Effectuer un audit mobile complet	MobSF	MobSF couvre plus de catégories : manifeste, permissions, certificat, trackers, composants et analyse dynamique.
Faire une démonstration pédagogique	SecretHunter Android	L'interface est simple et les résultats sont directement compréhensibles par un public étudiant.
Analyser le comportement runtime	MobSF	SecretHunter Android est statique dans sa version actuelle, tandis que MobSF propose une analyse dynamique.
Identifier rapidement un risque critique lié aux secrets	SecretHunter Android	Le score donne une priorité forte aux secrets critiques comme les tokens, mots de passe et clés API.
Préparer une analyse professionnelle complète	MobSF + SecretHunter Android	Les deux outils peuvent être utilisés ensemble : SecretHunter pour cibler les secrets/endpoints, MobSF pour l'audit global.

En résumé, SecretHunter Android possède une valeur ajoutée claire malgré son statut de prototype. Sa force n'est pas d'être plus complet que MobSF, mais d'être plus ciblé, plus lisible et mieux adapté à l'apprentissage progressif de l'analyse statique Android. L'outil permet de suivre tout le cheminement : APK importé, fichiers extraits, secrets détectés, endpoints identifiés, score calculé et recommandations proposées. Cette traçabilité rend le projet pertinent pour un contexte académique, tout en ouvrant la voie à des extensions futures vers une plateforme plus complète.

14. Évaluation expérimentale

14.1. Corpus de test

Trois APKs ont été utilisés : DIVA, OWASP UnCrackable Level 2 et InsecureBankv2. Ils ont été choisis car ils sont volontairement vulnérables ou pédagogiques, ce qui permet de tester des comportements différents : secrets exposés, absence de secrets, grand volume de fichiers, endpoints nombreux et application bancaire vulnérable.

TABLE 16 – Corpus de test et justification.

APK	Source	Objectif	Traçabilité
DIVA	Projet DIVA Android [7]	Tester secrets, mauvaise configuration et endpoints	<code>diva-beta.apk</code> ; SHA-256 à calculer avant dépôt final
UnCrackable L2	OWASP MAS Crackmes [8]	Tester extraction, reverse engineering et absence de secrets simples	<code>UnCrackable-Level2.apk</code> ; SHA-256 à calculer
InsecureBankv2	Dépôt InsecureBankv2 [9]	Tester une application bancaire vulnérable	<code>app-debug.apk</code> ; SHA-256 à calculer

14.2. Résultats observés

TABLE 17 – Résultats observés avec SecretHunter Android.

Métrique	DIVA	UnCrackable L2	InsecureBankv2
Nom de l'APK	<code>diva-beta.apk</code>	<code>UnCrackable-Level2.apk</code>	<code>app-debug.apk</code>
Fichiers analysés	3 482	1 972	10 680
Secrets détectés	6	0	24
Endpoints détectés	1 481	1 032	2 352
Score global	68/100	20/100	72/100
Risque global	Élevé	Faible	Élevé
Résultats critiques	6	0	24
Résultats faibles	1 481	1 032	2 352
Rapport JSON/HTML	Oui	Oui	Oui

14.3. Interprétation

DIVA est classée en risque élevé avec 6 secrets et 1481 endpoints. Ce résultat confirme l'utilité du scanner de secrets sur une application volontairement vulnérable. UnCrackable Level 2 obtient un score faible car aucun secret n'est détecté, malgré la présence d'endpoints. Ce résultat est cohérent avec sa nature de challenge de reverse engineering. InsecureBankv2 représente le cas le plus critique : 24 secrets, 2352 endpoints et 10680 fichiers analysés. Le score de 72/100 reflète la présence de nombreux secrets critiques.

Les résultats montrent que le score augmente principalement lorsque des secrets sont détectés. Les endpoints contribuent à la cartographie de l'application mais restent généralement moins critiques que les secrets.

15. Vérité terrain, faux positifs et limites de l'évaluation

La version actuelle valide le fonctionnement du pipeline, mais elle ne constitue pas encore une évaluation scientifique complète. Pour calculer précision, rappel et F1-score, il faut disposer d'une vérité terrain annotée : chaque résultat devrait être classé comme vrai positif, faux positif ou faux négatif.

Les endpoints sont la principale source de faux positifs. Le scanner détecte beaucoup de chaînes réseau, mais toutes ne sont pas exploitables. Les namespaces Android, imports Java

et chaînes de configuration doivent être filtrés. À l'inverse, certains secrets simples peuvent être difficiles à détecter si les règles sont trop strictes ou si les valeurs sont obfusquées.

TABLE 18 – Principales sources de faux positifs et corrections envisagées.

Source	Correction envisagée
Namespaces Android	Liste d'exclusion dédiée.
Imports Java/Android	Filtrage par préfixes <code>java.</code> , <code>android.</code> , <code>javax..</code>
Chaînes de test	Réduction du score si contexte <code>test/example/dummy</code> .
Données obfusquées	Analyse complémentaire par patterns et entropie.
Endpoints incomplets	Validation par protocole, domaine et contexte fichier.

16. Assurance qualité et reproductibilité

Les tests réalisés couvrent l'upload d'APK, l'extraction, l'appel des scanners, l'affichage frontend et la génération de rapports. Pour une assurance qualité plus robuste, la version finale doit inclure des tests unitaires pour `secret_scanner.py`, `endpoint_scanner.py` et `risk_scoring.py`, ainsi que des tests d'intégration pour `/api/analyze`, `/status` et `/results`.

TABLE 19 – Plan qualité recommandé.

Test	Objectif
Tests unitaires secrets	Vérifier que les règles détectent les secrets attendus.
Tests unitaires endpoints	Vérifier la détection et le filtrage des URLs/domaines/IP.
Tests unitaires scoring	Vérifier la formule 65/35 et les seuils de risque.
Tests API	Vérifier upload, statut, résultats et erreurs.
Tests frontend	Vérifier affichage des compteurs, tableaux et rapports.
CI GitHub Actions	Automatiser tests et linting à chaque push.

La reproductibilité repose sur `requirements.txt`, `package-lock.json`, les scripts PowerShell et la documentation. Un futur `docker-compose.yml` permettra de standardiser l'environnement et de réduire les différences entre machines.

17. Impact pédagogique et technique

SecretHunter Android a un impact pédagogique direct. Il permet de visualiser la structure interne d'un APK, de relier un résultat à un fichier et à une ligne, et d'expliquer les notions de secret codé en dur, endpoint exposé, faux positif et priorisation des risques. Contrairement à un outil boîte noire, il montre les étapes internes du pipeline et rend le processus compréhensible.

D'un point de vue technique, la plateforme sert de base pour ajouter de nouveaux scanners : analyse du manifeste, composants exportés, stockage local non sécurisé, WebView, configuration réseau, cryptographie faible et détection des permissions dangereuses. Elle peut également être intégrée dans un pipeline DevSecOps via API REST et export SARIF.

18. Limites et perspectives

Les limites actuelles sont les suivantes :

- l’analyse est statique et ne couvre pas le comportement runtime ;
- les règles de secrets reposent sur des expressions régulières ;
- les endpoints peuvent contenir des faux positifs ;
- la vérité terrain annotée n’est pas encore disponible ;
- l’export PDF/SARIF reste à finaliser ;
- le modèle IA local n’est pas encore obligatoire et doit rester optionnel ;
- les grands APKs peuvent nécessiter des optimisations de performance.

Les perspectives prioritaires sont :

- filtrage strict des namespaces Android et imports Java ;
- ajout d’une analyse dynamique avec MobSF ou émulateur ;
- export SARIF pour intégration CI/CD ;
- conteneurisation Docker ;
- modèle IA local pour recommandations plus variées ;
- corpus annoté pour calculer précision, rappel et F1-score ;
- tableau de bord historique avec plusieurs analyses.

19. Disponibilité et réutilisation

La version exécutable de SecretHunter Android est disponible à partir du dépôt GitHub du projet. Le backend se lance avec `python -m backend.app` et le frontend avec `npm.cmd run dev`. L’API est accessible via <http://127.0.0.1:5000> et l’interface web via <http://localhost:5173>.

Avant archivage final, l’équipe doit publier le tag `v1.0.0`, ajouter ou vérifier le fichier `LICENSE`, calculer les empreintes SHA-256 des APKs du corpus, compléter la documentation et préparer une capsule Docker ou justifier son absence.

20. Conclusion

SecretHunter Android est une plateforme académique modulaire dédiée à l’analyse statique des APK Android. Elle combine extraction, détection de secrets, identification d’endpoints, scoring du risque, recommandations et génération de rapports. Les résultats obtenus sur DIVA, OWASP UnCrackable Level 2 et InsecureBankv2 montrent que le pipeline est opérationnel et que le score reflète la présence ou l’absence de secrets critiques.

La comparaison avec MobSF montre que SecretHunter Android est particulièrement adapté à une analyse ciblée secrets/endpoints et à un usage pédagogique. MobSF reste plus complet pour une évaluation mobile globale. Les deux approches sont donc complémentaires.

Les prochaines étapes concernent principalement la réduction des faux positifs, l’ajout de tests automatisés, la constitution d’un corpus annoté, l’amélioration de la reproductibilité et l’extension vers PDF, SARIF, Docker et analyse dynamique.

Remerciements

Les auteurs remercient leur encadrant ainsi que l’équipe pédagogique de l’ENSA Marrakech pour l’accompagnement de ce projet.

Références

- [1] OWASP Foundation. *Mobile Application Security Verification Standard (MASVS)*. OWASP Mobile Application Security, 2024–2026. <https://mas.owasp.org/MASVS/>. Consulté le 17 mai 2026.
- [2] OWASP Foundation. *Mobile Application Security Testing Guide (MASTG)*. OWASP Mobile Application Security, 2024–2026. <https://mas.owasp.org/MASTG/>. Consulté le 17 mai 2026.
- [3] OpenSecurity. *Mobile Security Framework MobSF Documentation*. 2026. <https://mobsf.github.io/docs/>. Consulté le 17 mai 2026.
- [4] Skylot. *JADX : Dex to Java decompiler*. GitHub, 2026. <https://github.com/skylot/jadx>. Consulté le 17 mai 2026.
- [5] iBotPeaches. *Apktool : A tool for reverse engineering Android apk files*. 2026. <https://apktool.org/>. Consulté le 17 mai 2026.
- [6] OASIS. *Static Analysis Results Interchange Format (SARIF) Version 2.1.0*. 2020. <https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html>. Consulté le 17 mai 2026.
- [7] Payatu. *DIVA Android : Damn Insecure and Vulnerable App*. GitHub. <https://github.com/payatu/diva-android>. Consulté le 17 mai 2026.
- [8] OWASP Foundation. *MAS Crackmes : Android UnCrackable Level 2*. <https://mas.owasp.org/crackmes/>. Consulté le 17 mai 2026.
- [9] Dinesh Shetty. *Android InsecureBankv2 : Vulnerable Android application*. GitHub. <https://github.com/dineshshetty/android-insecurebankv2>. Consulté le 17 mai 2026.
- [10] S. Arzt et al. *FlowDroid : Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps*. PLDI, 2014.
- [11] W. Enck et al. *TaintDroid : An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones*. OSDI, 2010.
- [12] M. Meli, M. R. McNiece, and B. Reaves. *How Bad Can It Get ? Characterizing Secret Leakage in Public GitHub Repositories*. NDSS, 2019.

Annexe A. Pitch de démonstration

SecretHunter Android est une plateforme web d’analyse statique d’APKs Android. L’objectif est de détecter rapidement les secrets codés en dur, les endpoints exposés et le niveau de risque global d’une application. L’utilisateur importe un APK depuis l’interface React ; le backend Flask valide le fichier, l’extraît avec JADX et Apktool, applique les scanners, calcule un score de risque et génère un rapport JSON/HTML. Les tests sur DIVA, UnCrackable Level 2 et InsecureBankv2 montrent que le pipeline fonctionne et que le score augmente lorsque des secrets critiques sont présents. Le projet est pédagogique, modulaire et extensible vers l’analyse dynamique, SARIF, Docker et CI/CD.